ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA

Corso di Laurea in Ingegneria e Scienze Informatiche

Sicurezza Kernel-Level in ambienti Cloud-Native: il caso di studio Falco e Tetragon

Tesi di laurea in:
Virtualizzazione e Integrazione di Sistemi

Relatore
Prof. Vittorio Ghini

Candidato
Matteo Grandini

II Sessione di Laurea
Anno Accademico 2025-2026

Ringraziamenti

Desidero ringraziare il Prof. Vittorio Ghini per la supervisione e il supporto durante lo sviluppo di questa tesi.

Infine, ringrazio i miei genitori, mia sorella, i miei nonni e tutti gli amici che mi hanno sostenuto in questo percorso.

Sommario

Indice

Sommario **v**

1 Introduzione **1**

2 Background **3**
2.1 Sicurezza Cloud-Native 3
2.2 Fondamenti di eBPF 4

3 Falco e Tetragon **7**
3.1 Falco 7
3.1.1 Architettura di Falco 7
3.2 Tetragon 9
3.2.1 Architettura di Tetragon 9

4 Casi d'uso e Deployment **11**
4.1 Casi d'uso principali 11
4.1.1 Punti di forza di Falco 12
4.1.2 Punti di forza di Tetragon 12
4.2 Requisiti di sistema e kernel 13
4.3 Deployment: Docker e Kubernetes 13
4.3.1 Deployment in container Docker standalone 13
4.3.2 Deployment in Kubernetes 14
4.4 Facilità di configurazione e gestione 15
4.4.1 Confronto sulle definizioni delle policy 15
4.4.2 Risultati del confronto operativo 19

5 Testing automatizzato **23**
5.1 Motivazione e Obiettivi 23
5.2 Metriche di valutazione 24
5.3 Configurazione dell'ambiente di test 25
5.3.1 Selezione dei test 27

5.3.2 Formato della suite di test 29
5.3.3 Definizione delle TracingPolicy 30
5.4 Progettazione del programma di testing 31
5.4.1 Architettura 31
5.4.2 Isolamento dei test e correlazione degli eventi 36

6 Risultati e Discussione **39**
6.1 Risultati della suite senza stress 39
6.1.1 Efficacia della detection 39
6.1.2 Latenza di rilevazione 40
6.1.3 Costo computazionale 40
6.2 Risultati della suite di stress 42
6.2.1 Perdita di eventi 43
6.3 Discussione complessiva 45

7 Conclusione e sviluppi futuri **47**

A Dichiarazione sull'uso di strumenti di GenAI **51**
A.1 Strumenti utilizzati 51
A.2 Esempi di Prompt 51

53

Bibliografia **53**

Elenco delle figure

2.1 Architettura di eBPF. (Fonte immagine: [4]) 5

3.1 Architettura di Falco. (Fonte immagine: [11]) 8
3.2 Architettura di Tetragon. (Fonte immagine: [5]) 10

5.1 Sequenza di esecuzione del programma di testing automatizzato. . . 33
5.2 Diagramma delle classi del programma di testing automatizzato. . . 34

6.1 Utilizzo medio della CPU nella suite `all-no-stress`. 41
6.2 Utilizzo massimo della memoria nella suite `all-no-stress`. 41
6.3 Utilizzo medio della CPU durante i test di stress. 42
6.4 Utilizzo massimo della memoria durante i test di stress. 43
6.5 Eventi persi rilevati durante i test di stress. 44

Elenco dei listati

4.1 Esempi di regole Falco. 15
4.2 TracingPolicy per l'esecuzione di shell. 17
4.3 TracingPolicy per la scrittura nella directory `/etc/`. 17
4.4 TracingPolicy per l'esecuzione da percorsi scrivibili. 18
5.1 Docker Compose per la creazione dei servizi Falco e Tetragon. . . . 26
5.2 Costruzione dell'immagine Docker contenente il binario `goart`. . . . 28
5.3 Esempi di test utilizzati nella campagna sperimentale. 29
5.4 TracingPolicy per la rilevazione della connessione TCP. 31
5.5 Interrogazione di Prometheus per i contatori degli eventi persi. . . . 35

Capitolo 1

Introduzione

Nel contesto della trasformazione digitale, la diffusione di applicazioni containerizzate e di architetture cloud-native ha reso i sistemi software sempre più dinamici, scalabili e distribuiti. La stessa evoluzione che rende moderne le infrastrutture di produzione, però, introduce nuove sfide di sicurezza: i workload condividono lo stesso kernel dell'host, le risorse sono altamente volatili e gli attacker possono sfruttare comportamenti apparentemente innocui a livello runtime per ottenere privilege escalation, accesso non autorizzato o esfiltrazione di dati. In un ecosistema in cui la velocità di deployment e la complessità operativa crescono in modo esponenziale, la sicurezza non può essere pensata solo come controllo statico delle immagini o verifica del codice sorgente: occorre una visibilità continua del comportamento dei processi, delle syscall e delle risorse del sistema, anche dopo l'avvio dell'applicazione. È in questo scenario che strumenti di runtime security come Falco e Tetragon assumono un ruolo centrale, offrendo meccanismi per osservare, rilevare e, in alcuni casi, contrastare attività sospette in tempo reale.

Il punto centrale della presente tesi è comprendere se e in che misura strumenti di sicurezza kernel-level basati su eBPF possano rappresentare una risposta efficace ai rischi della sicurezza nei container e nei cluster Kubernetes, senza compromettere l'efficienza operativa e la semplicità di gestione. In particolare, la domanda di ricerca che guida il lavoro è la seguente: *in che modo Falco e Tetragon differiscono nella loro capacità di rilevare e mitigare attività sospette in ambienti cloud-native, e quali sono i trade-off tra precisione, latenza e complessità di configurazione?*

L'obiettivo del progetto è quindi di confrontare in modo critico due strumenti emblematici del panorama attuale: Falco, orientato principalmente al rilevamento e all'alerting, e Tetragon, progettato per un modello più vicino al kernel e alla mitigazione in tempo reale. La ricerca si sviluppa quindi con un approccio di tipo comparativo e empirico: vengono esaminati i principi di funzionamento, i modelli di policy, i requisiti di installazione e i pattern di deployment in ambienti Docker e Kubernetes; successivamente, vengono valutate configurazioni rappresentative di scenari di sicurezza quali spawn di shell interattive, scritture in percorsi sensibili e esecuzione di binari non autorizzati. Questo approccio consente di mettere in relazione le caratteristiche teoriche degli strumenti con la loro applicazione pratica, evidenziando i vantaggi, i limiti e i compromessi che emergono in contesti realistici.

La tesi è organizzata in una sequenza logica che parte dai fondamenti teorici e arriva alla valutazione operativa degli strumenti. In primo luogo, viene introdotto il contesto della sicurezza cloud-native e le basi di eBPF (Capitolo 2); in seguito, vengono descritti architettura, contesti di deployment e modalità di funzionamento di Falco e Tetragon (Capitoli 3 e 4); infine, viene presentata la progettazione di un programma per confrontare le capacità di entrambi gli strumenti e l'analisi dei derivanti risultati (Capitoli 5 e 6). In questo modo, il lavoro non si limita a una descrizione formale dei due sistemi, ma costruisce un percorso che collega modello concettuale, scenario applicativo e evidenza sperimentale, offrendo al lettore una visione completa delle opportunità e dei limiti della sicurezza kernel-level nei moderni ambienti cloud-native.

Dichiarazione sull'uso di strumenti di GenAI Per la redazione di questa tesi, sono stati utilizzati strumenti di intelligenza artificiale generativa (GenAI) quali *GitHub Copilot* e *Gemini 3.1 Pro* per supporto nella scrittura del codice e nella revisione dei testi. I dettagli sul loro utilizzo sono descritti nell'Appendice A.

Capitolo 2

Background

Questo capitolo fornisce il quadro concettuale necessario per comprendere il ruolo di strumenti come Falco e Tetragon nella sicurezza di ambienti cloud-native. Vengono introdotti due elementi centrali: le caratteristiche principali della sicurezza nei container e il funzionamento di eBPF come tecnologia di osservabilità e controllo a livello kernel.

2.1 Sicurezza Cloud-Native

Il paradigma cloud-native si è ormai consolidato come standard per lo sviluppo e il deployment di applicazioni moderne, grazie alla sua capacità di scalare dinamicamente e di sfruttare al meglio le risorse distribuite. Tuttavia, questa architettura introduce complessità significative in termini di sicurezza. La natura effimera dei container e la frequente interazione tra servizi diversi rendono difficile mantenere una visibilità completa e un controllo granulare sul sistema.

I container condividono lo stesso kernel dell'host e molte attività sensibili, come privilege escalation, manipolazione di filesystem montati o esfiltrazione di dati, si manifestano attraverso syscall e comportamenti runtime che non sono immediatamente evidenti osservando soltanto i log applicativi. Per questo motivo, la sicurezza moderna nel paradigma cloud-native si fonda su tre pilastri principali:

- **Riduzione della superficie d'attacco:** uso del principio del privilegio minimo, immutabilità delle immagini e meccanismi di namespace e cgroups

per isolare risorse.

- **Pre-deployment security:** scansione delle immagini e verifica di conformità durante la pipeline CI/CD.
- **Runtime security e osservabilità:** monitoraggio continuo del comportamento dei processi e delle risorse, correlazione di eventi e capacità di risposta automatica o manuale.

In particolare, l'osservabilità a livello runtime richiede strumenti in grado di collegare eventi di alto livello, come richieste di rete, creazione di processi o accesso a file sensibili, con le entità del cluster, come pod, container e namespace.

Dal punto di vista operativo, una strategia completa di sicurezza cloud-native dovrebbe combinare controlli preventivi con strumenti di rilevamento e risposta a runtime capaci di correlare segnali multipli e, quando necessario, applicare enforcement a bassa latenza.

Nell'ambito di questa tesi, l'attenzione è posta sulla parte di runtime security, esplorando come strumenti come Falco e Tetragon possano essere utilizzati per ottenere visibilità, rilevazione e mitigazione di comportamenti sospetti o malevoli in ambienti containerizzati.

2.2 Fondamenti di eBPF

eBPF (extended Berkeley Packet Filter) è un'estensione del meccanismo BPF (Berkeley Packet Filter) che consente l'esecuzione di programmi sandboxed all'interno del kernel Linux. Questi programmi possono essere utilizzati per monitorare e, in alcuni casi, influenzare il comportamento del sistema operativo in tempo reale, senza dover modificare il codice del kernel o caricare moduli esterni. [4]

Dal punto di vista architetturale, un programma eBPF viene caricato in risposta a specifici hook del kernel, come kprobe, uprobe, tracepoint, socket filter e programmi di rete. Quando viene attivato, il programma può leggere o scrivere dati in apposite strutture chiamate mappe, scambiare informazioni con lo user-space[1] e, se necessario, modificare il flusso di esecuzione di un evento. Le

[1] Per "user-space" si intende lo spazio di esecuzione dell'utente, in contrapposizione al "kernel-space".

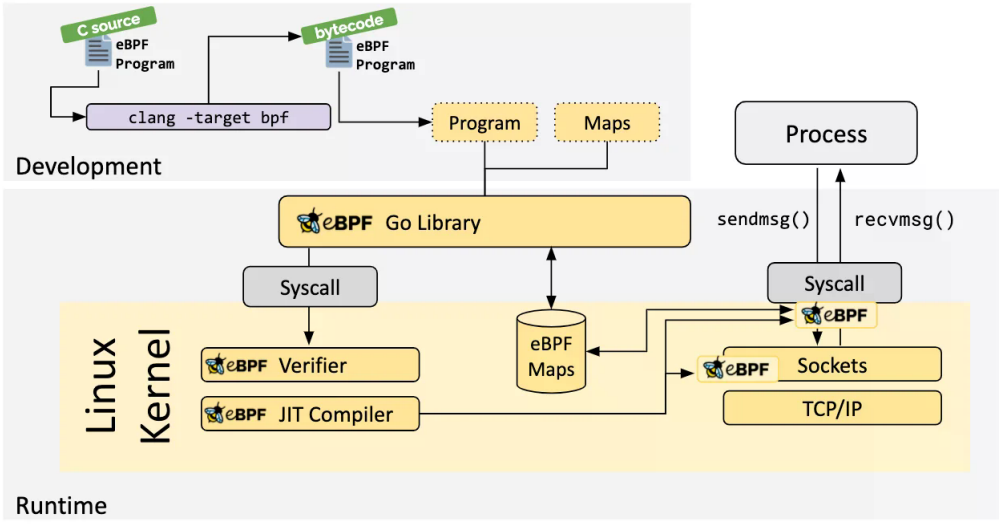


Figura 2.1: Architettura di eBPF. (Fonte immagine: [4])

mappe rappresentano uno degli elementi chiave dell'architettura di eBPF, consentendo di memorizzare lo stato, aggregare metriche e condividere informazioni tra il programma kernel e un agente nello user-space. [4]

La sicurezza del programma eBPF è garantita dal Verifier, un componente all'interno del kernel che analizza il programma prima dell'esecuzione per verificarne la terminazione, l'assenza di loop infiniti e l'accesso corretto alle risorse. Questa architettura rende eBPF particolarmente adatto a scenari in cui è necessario ottenere visibilità granulare sul comportamento del sistema senza il costo di agenti user-space tradizionali. La presenza di mappe condivise, helper stabili e compilazione Just-In-Time (JIT) consente di costruire pipeline di telemetria efficienti, mentre l'aggancio a hook diversi permette di intercettare eventi a livelli di astrazione differenti, dalle syscall a punti di tracing più specifici. [4]

L'architettura di eBPF può essere riassunta nella Figura 2.1, che mostra in modo sintetico il percorso di un programma eBPF: il codice sorgente viene compilato in bytecode, caricato nel kernel, verificato dal Verifier e infine compilato in codice macchina dal JIT Compiler. A questo punto, il programma può reagire agli eventi del kernel, aggiornare le mappe e comunicare con lo user-space. [4]

Capitolo 3

Falco e Tetragon

In questo capitolo vengono presentati due strumenti di sicurezza runtime per ambienti cloud-native: Falco e Tetragon. Entrambi sfruttano le capacità di osservabilità offerte da eBPF, ma adottano approcci differenti per la rilevazione e la mitigazione di comportamenti sospetti o malevoli.

3.1 Falco

Falco è una piattaforma di runtime security orientata principalmente al rilevamento. Il suo obiettivo è osservare i comportamenti del sistema, correlare gli eventi con delle poilcy e generare alert quando viene riconosciuto un pattern anomalo o potenzialmente dannoso. La filosofia di Falco è quella di fornire una visibilità ampia e facilmente integrabile con i sistemi di sicurezza, alerting e incident response già esistenti. [17]

3.1.1 Architettura di Falco

L'architettura di Falco è organizzata in una pipeline di raccolta, normalizzazione e valutazione delle policy. A livello di input, il sistema acquisisce eventi provenienti da sorgenti diverse: syscall del kernel, eventi di container runtime, metadati di orchestrazione e, in alcune configurazioni, dati provenienti da plugin esterni. La raccolta avviene mediante un componente di acquisizione che permette di esporre un flusso uniforme di eventi, indipendentemente dalla loro origine concreta. In pra-

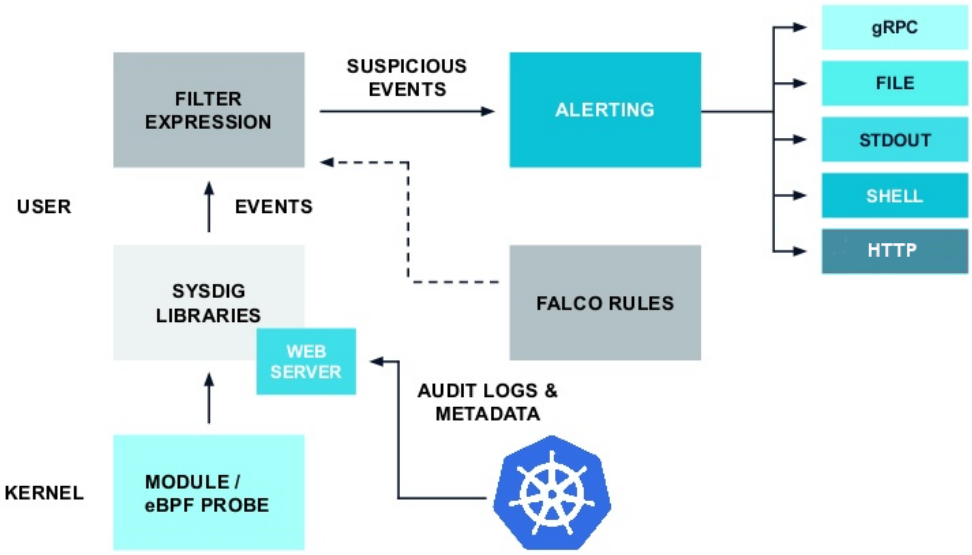


Figura 3.1: Architettura di Falco. (Fonte immagine: [11])

tica, il kernel continua a essere la fonte primaria del segnale, ma Falco arricchisce l'evento con contesto applicativo e di orchestrazione. [10, 16, 19]

L'evento non viene valutato direttamente nel kernel, ma viene inviato a un demone user-space, dove il motore di regole verifica le condizioni definite nelle policy. Le regole di Falco sono formulate in YAML e permettono di specificare le condizioni di attivazione, l'output da produrre e il livello di priorità. Quando una di queste regole viene soddisfatta, Falco genera un alert strutturato e lo inoltra a uno o più output configurati. [10, 12]

Questa progettazione presenta vantaggi chiari in termini di semplicità operativa e compatibilità, ma implica anche una logica prettamente reattiva. L'evento viene catturato e valutato dopo che la syscall o l'azione critica è già stata eseguita; quindi Falco è ideale per analisi forense, monitoraggio continuo, alerting e automazione, ma non per bloccare l'azione nel punto esatto in cui si verifica. La sua architettura è quindi particolarmente efficace in scenari in cui il requisito principale è la visibilità ambientale e la capacità di trasformare i segnali di sistema in azioni di risposta organizzate.

3.2 Tetragon

Tetragon è un progetto di Cilium[1] dedicato all'osservabilità e al runtime enforcement a livello kernel. A differenza di Falco, il modello di Tetragon non si limita a osservare gli eventi per poi valutarli fuori dal percorso di esecuzione, ma cerca di integrare il controllo della policy direttamente nel flusso dell'operazione da monitorare. Il suo obiettivo è ridurre la finestra temporale tra l'osservazione di una condotta sospetta e l'applicazione di una risposta, minimizzando la possibilità che un'azione dannosa produca effetti permanenti prima di essere fermata.

Tetragon osserva processi, syscall, file, rete e contesto di esecuzione del workload, correlandoli con le informazioni Kubernetes e con l'identità del pod. Quando una policy viene violata, il sistema non si limita a produrre un avviso: può generare un'azione di mitigazione immediata, ad esempio interrompendo un processo o bloccando un'operazione prima che compia un effetto significativo.

3.2.1 Architettura di Tetragon

L'architettura di Tetragon è progettata per essere kernel-first. L'agente viene eseguito su ciascun nodo e carica programmi eBPF in risposta a hook del kernel, come kprobe, tracepoint e altri punti di osservazione del sistema. Questo permette di avere un monitoraggio efficiente con costo ridotto di trasferimento dei dati verso user-space. L'idea è che la decisione di raccolta e filtraggio avvenga il più vicino possibile alla sorgente dell'evento, senza far transitare tutti i dati attraverso un layer applicativo intermedio. [24, 25]

Il percorso degli eventi di Tetragon è quindi strutturato in più livelli. In primo luogo, i programmi eBPF vengono agganciati ai punti di tracciamento rilevanti per osservare l'esecuzione dei processi e le chiamate di sistema sensibili. In secondo luogo, i dati vengono filtrati e aggregati tramite mappe e strutture kernel-side per mantenere lo stato necessario alla policy. In terzo luogo, le informazioni vengono consegnate a un componente user-space che espone il risultato della valutazione, integra il contesto Kubernetes e gestisce le azioni di risposta. In questo modello, la parte di enforcement può essere applicata direttamente nel kernel, riducendo no-

[1] Cilium è un progetto open source per networking e sicurezza in ambienti cloud-native [2].

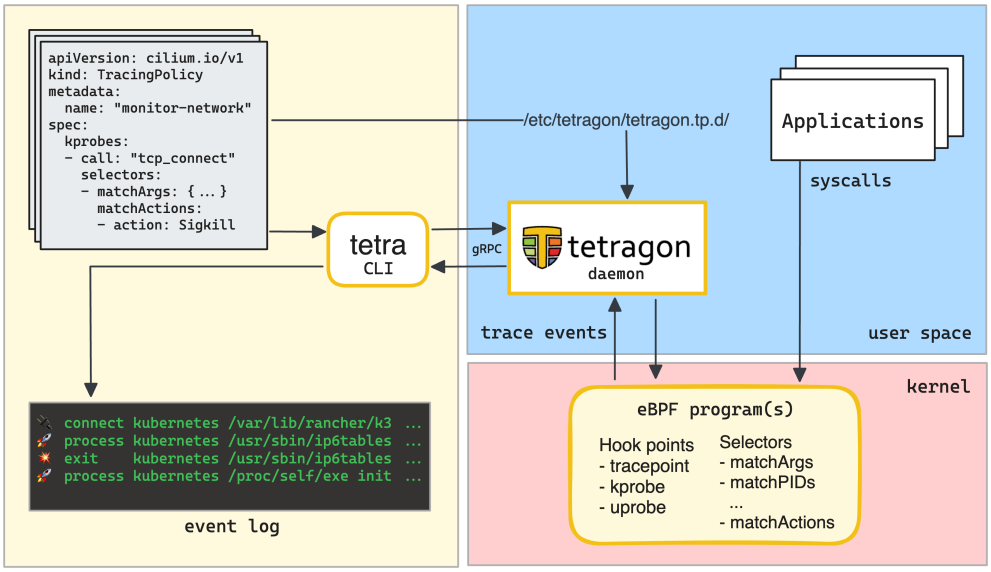


Figura 3.2: Architettura di Tetragon. (Fonte immagine: [5])

tevolmente la latenza rispetto a un'architettura basata esclusivamente su alerting. [24]

Un elemento importante dell'architettura di Tetragon è la forte dipendenza dal contesto del sistema operativo e dall'uso di metadati del kernel, tra cui il supporto a BTF[2]. BTF consente al programma eBPF di accedere in modo più robusto a strutture e tipi di dati del kernel, migliorando l'affidabilità del tracing e la portabilità tra kernel con configurazioni simili. Questa caratteristica rende Tetragon particolarmente adatto a ambienti Linux moderni in cui sono richieste policy precise e sensibili al contesto del workload. [24, 25]

[2]BTF (BPF Type Format) serve a descrive i tipi di dati in un programma BPF/eBPF. [20]

Capitolo 4

Casi d'uso e Deployment

Questo capitolo descrive i principali casi d'uso di Falco e Tetragon, evidenziando le differenze operative e le aree di applicazione. Vengono inoltre discussi i requisiti di sistema e kernel, nonché le modalità di deployment più comuni, con particolare attenzione a host singoli, container Docker standalone e cluster Kubernetes.

4.1 Casi d'uso principali

Pur condividendo la stessa area applicativa, Falco e Tetragon risultano ottimizzati per esigenze parzialmente diverse. Entrambi possono essere adottati in diversi scenari comuni di runtime security, nei quali è necessario osservare il comportamento dei workload e collegare gli eventi di sistema al contesto dell'ambiente cloud-native. I principali casi d'uso condivisi sono:

- **Rilevazione di attività anomale nei container:** osservazione di shell interattive inattese, esecuzione di binari non previsti, accessi a path sensibili e modifiche sospette al filesystem.
- **Verifica di conformità:** verifica del rispetto di policy runtime, ad esempio relativamente all'uso di privilegi elevati, capability non autorizzate o risorse sensibili.
- **Osservabilità di processi e syscall:** raccolta di eventi a livello kernel e correlazione con processi, container, pod, namespace e altri metadati.

- **Supporto all'analisi e alla risposta agli incidenti:** produzione di telemetria utile per ricostruire una sequenza di eventi e attivare procedure di risposta manuali o automatizzate.

4.1.1 Punti di forza di Falco

Falco è particolarmente efficace quando il requisito dominante è la detection, l'auditing e l'integrazione con processi SOC/SIEM. Alcuni casi d'uso specifici in cui Falco eccelle sono:

- **Alerting centralizzato:** inoltro degli eventi verso syslog, webhook, piattaforme SIEM o pipeline di incident response automatizzata.
- **Analisi post-evento:** ricostruzione delle sequenze di eventi grazie a regole descrittive e output strutturati.
- **Tuning di regole descrittive:** ampia possibilità di personalizzazione delle condizioni di rilevamento per adattarle ai workload e ridurre falsi positivi.

[13, 17]

4.1.2 Punti di forza di Tetragon

Tetragon è preferibile nei casi in cui, oltre alla visibilità, sia richiesta una riduzione della finestra di rischio attraverso enforcement tempestivo e filtraggio vicino alla sorgente dell'evento. I casi d'uso specifici sono:

- **Prevenzione di azioni ad alto impatto:** blocco o terminazione di processi in presenza di condizioni vietate dalle policy.
- **Enforcement repentino:** applicazione della risposta nel percorso di esecuzione, con l'obiettivo di ridurre la latenza tra rilevazione e mitigazione.
- **Riduzione del rumore di telemetria:** selezione più precisa degli eventi rilevanti prima della consegna a user-space.

[24, 25]

In sintesi, entrambi gli strumenti possono fornire visibilità sul comportamento runtime; tuttavia, Falco è principalmente orientato alla generazione di alert, mentre Tetragon aggiunge meccanismi di enforcement più vicini alla sorgente dell'evento.

4.2 Requisiti di sistema e kernel

Entrambi gli strumenti operano su Linux e richiedono accesso alle funzionalità del kernel necessarie alla sorgente di eventi e alle policy selezionate. Il supporto eBPF e i meccanismi di caricamento e validazione dei programmi sono essenziali per le modalità basate su eBPF; i requisiti esatti dipendono tuttavia dalla versione del kernel, dal driver scelto e dagli hook utilizzati. Sono inoltre necessari permessi adeguati nella forma di privilegi elevati oppure in un insieme più ristretto di capability e profili di sicurezza. [15, 14, 23, 26]

Falco può essere eseguito con diverse strategie di acquisizione eventi (driver kernel o eBPF probe), il che offre maggiore flessibilità di compatibilità su ambienti eterogenei. Tetragon, invece, dipende in modo più marcato dalle capacità moderne del kernel, in particolare dal supporto a BTF per un tracciamento robusto e portabile. [14, 25, 26]

4.3 Deployment: Docker e Kubernetes

La scelta della modalità di deployment influenza profondamente visibilità, copertura e costo operativo. È perciò importante conoscere le differenze tra le due modalità principali: esecuzione come container Docker standalone e deployment in un cluster Kubernetes.

4.3.1 Deployment in container Docker standalone

Il deployment standalone consiste nell'eseguire l'agente di sicurezza come container Docker sul nodo da osservare, senza introdurre un orchestratore che ne gestisca il

ciclo di vita. Il modello è adatto a host singoli, ambienti di laboratorio e installazioni in cui si desidera mantenere il numero di componenti ridotto. Offre rapidità di prova e un controllo diretto sui volumi, sui namespace e sui parametri del processo; tuttavia, non fornisce automaticamente alta disponibilità, aggiornamenti coordinati o altre funzionalità tipiche di un orchestratore.

Poiché il container dell'agente deve osservare processi e strutture del kernel appartenenti all'host, l'isolamento standard di Docker deve essere parzialmente ridotto. In particolare, sono necessari l'accesso al namespace PID dell'host, ai cgroups e, a seconda della sorgente di eventi utilizzata, ad alcune directory di sistema che espongono informazioni sul kernel e sui container. Tetragon deve poter caricare e agganciare i programmi eBPF e per fare ciò, nel setup tradizionale, utilizza il file `vmlinux` contenente le informazioni BTF del kernel. I privilegi possono essere concessi tramite `privileged`, oppure mediante un insieme più restrittivo di capability e mount, purché siano sufficienti per la configurazione scelta. [18, 23, 26]

4.3.2 Deployment in Kubernetes

In un cluster Kubernetes, Falco e Tetragon vengono normalmente distribuiti mediante un *DaemonSet*, così che ogni nodo esegua una istanza capace di osservare i workload locali. Il chart o i manifest definiscono il ciclo di vita del pod, i mount, le configurazioni e l'esposizione dei log; Kubernetes può quindi occuparsi di scheduling, riavvio e aggiornamento dell'agente. Entrambi gli strumenti mettono a disposizione una serie di strumenti e librerie per facilitare l'installazione e la configurazione. [15, 23]

Il vantaggio principale rispetto al container standalone è operativo: la configurazione può essere versionata come risorsa Kubernetes e replicata e aggiornata automaticamente sui nodi del cluster. Falco beneficia dell'integrazione con chart, regole e output verso sistemi esterni; Tetragon si integra naturalmente con le identità dei pod e con l'ecosistema Cilium, rendendo più dirette le policy basate sul contesto Kubernetes. In entrambi i casi, l'orchestratore semplifica la gestione, ma non elimina la necessità di verificare la compatibilità del kernel di ciascun nodo. In un cluster eterogeneo, infatti, le differenze di BTF, hook o configurazione eBPF possono infatti produrre una copertura non uniforme.

4.4 Facilità di configurazione e gestione

La facilità di adozione di uno strumento di runtime security non dipende soltanto dal numero di comandi necessari per avviarlo. È necessario considerare anche la complessità della scrittura delle policy, il livello di integrazione con altri strumenti, le attività di tuning e la gestione delle modifiche nel tempo.

Al fine di valutare concretamente i due strumenti, una parte importante di questa tesi è stata rivolta all'installazione, alla configurazione e al tuning di Falco e Tetragon in ambienti Docker e Kubernetes. Sono stati considerati diversi scenari, dalla semplice osservazione di eventi a policy più complesse con enforcement, per comprendere le differenze operative e i requisiti di gestione.

Per eseguire il cluster Kubernetes di test è stato utilizzato `minikube` con driver Docker su nodo singolo, che ha permesso di simulare un ambiente di orchestrazione senza introdurre complessità aggiuntive. [21]

Si precisa, infine, che in questa sezione Falco e Tetragon vengono confrontati esclusivamente in termini di facilità di installazione, configurazione e gestione operativa, poiché la valutazione dell'efficacia delle policy e delle performance è oggetto del Capitolo 5.

4.4.1 Confronto sulle definizioni delle policy

La configurazione delle policy rappresenta il punto in cui le capacità di osservazione dei due strumenti vengono trasformate in decisioni di sicurezza. In Falco una regola combina generalmente una condizione su degli eventi, un messaggio di output, una priorità e un insieme di tag. La condizione può utilizzare sia il tipo di operazione sia il contesto arricchito del processo e del container. Il Listato 4.1 contiene tre regole rappresentative: la rilevazione di una shell interattiva, la scrittura sotto una directory sensibile e l'esecuzione di un binario introdotto nel layer scrivibile del container.

Listing 4.1: Esempi di regole Falco.

```
- rule: Interactive Shell in Container
  desc: Detect an interactive shell started inside a container
  condition: >
    spawned_process
    and container
```

```
    and proc.name in (bash, sh, dash)
    and proc.tty != 0
  output: >
    Interactive shell spawned in container
    (container=%container.name proc_exe=%proc.exe command=%proc.cmdline ...)
  priority: NOTICE
  tags: [container, shell, execution]
################################################################################
- rule: Write Below Sensitive Directory
  desc: Detect a write operation below a sensitive path
  condition: >
    open_write
    and container
    and fd.name startswith /etc/
  output: >
    File opened for writing below a sensitive directory
    (container=%container.name proc_exe=%proc.exe command=%proc.cmdline ...)
  priority: WARNING
  tags: [container, filesystem]
################################################################################
- rule: Drop and execute new binary in container
  desc: >
    Detect if an executable not belonging to the base image of a container is
        being executed.
  condition: >
    spawned_process
    and container
    and proc.is_exe_upper_layer=true
  output: >
    Executing binary not part of base image
    (container=%container.name proc_exe=%proc.exe command=%proc.cmdline ...)
  priority: CRITICAL
  tags: [container, process, mitre_persistence, TA0003]
```

La prima regola combina la creazione di un processo, l'appartenenza a un container, il nome della shell e la presenza di una TTY. Il campo `proc.tty` permette quindi di distinguere, una shell interattiva da un processo che utilizza lo stesso interprete per eseguire un comando non interattivo, potenzialmente riducendo drasticamente il numero di falsi positivi. La seconda regola utilizza invece il campo del file descriptor per limitare la rilevazione alle aperture in scrittura solo in `/etc/`. La terza usa il campo semantico `proc.is_exe_upper_layer`: in un container basato su OverlayFS, il valore è vero quando l'eseguibile proviene dall'upper layer scrivibile del container, cioè da una modifica o da un file aggiunto durante l'esecuzione, e non dai layer immutabili dell'immagine di base. In questo caso

Falco può quindi rilevare l'esecuzione di un binario introdotto nel container senza dover dedurre la sua origine soltanto dal percorso.

Tetragon esprime intenzioni analoghe attraverso risorse YAML *TracingPolicy*. Una policy specifica l'hook del kernel, gli argomenti da estrarre e i selettori che devono essere soddisfatti.

Listing 4.2: TracingPolicy per l'esecuzione di shell.

```yaml
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "detect-shell-execution"
spec:
  podSelector: {}
  kprobes:
  - call: "sys_execve"
    syscall: true
    args:
    - index: 0
      type: "string"
    selectors:
    - matchArgs:
      - index: 0
        operator: "Equal"
        values:
        - "/bin/bash"
        - "/usr/bin/bash"
        - "/bin/sh"
        - "/usr/bin/sh"
```

Listing 4.3: TracingPolicy per la scrittura nella directory `/etc/`.

```yaml
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "detect-etc-write"
spec:
  kprobes:
  - call: "security_file_permission"
    syscall: false
    return: true
    args:
    - index: 0
      type: "file"
    - index: 1
      type: "int"
    returnArg:
      index: 0
```

```
      type: "int"
    selectors:
    - matchArgs:
      - index: 0
        operator: "Prefix"
        values:
        - "/etc/"
      - index: 1
        operator: "Equal"
        values:
        - "2" # MAY_WRITE: 0x02
```

Listing 4.4: TracingPolicy per l'esecuzione da percorsi scrivibili.

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "block-execution-in-writable-paths"
spec:
  kprobes:
  - call: "sys_execve"
    syscall: true
    args:
    - index: 0
      type: "string"
    selectors:
    - matchArgs:
      - index: 0
        operator: "Prefix"
        values:
        - "/tmp/"
        - "/dev/shm/"
        - "/var/tmp/"
        - "/run/"
    - matchActions:
      - action: Sigkill
```

Nella prima policy (Listato 4.2) si può vedere l'utilizzo del filtro `podSelector` per escludere i processi dell'host dall'applicazione della regola. L'hook `sys_execve` permette di osservare l'esecuzione di un processo e di estrarre il percorso del binario eseguito, che viene confrontato con il selettore `matchArgs` specificato.

La policy per il rilevamento di scritture in `/etc/` (Listato 4.3) utilizza l'hook `security_file_permission` che permette di osservare diverse operazioni sui file, invece che una singola syscall. Il filtro `matchArgs` seleziona le operazioni con flag `MAY_WRITE` e percorso sotto `/etc/`.

Per quanto riguarda la terza policy (Listato 4.4), invece, la differenza rispetto a falco è molto più sostanziale: mentre Falco può verificare direttamente se l'eseguibile proviene dall'upper layer del filesystem overlay, Tetragon non riceve un attributo equivalente attraverso `sys_execve`. In questo caso, la policy Tetragon si basa su un'approssimazione dei percorsi scrivibili, come `/tmp/`, `/dev/shm/`, `/var/tmp/` e `/run/`. Ciò significa che la policy di Tetragon rileva l'esecuzione di binari in directory considerate scrivibili, ma non tutti i binari aggiunti all'upper layer e, viceversa, può rilevare un file legittimo collocato in una di quelle directory.

Le tre coppie di policy mostrano differenze di espressività. La sintassi Falco è più strutturata e intuitiva per chi deve combinare evento, contesto, output e severità in una regola leggibile. La sintassi Tetragon richiede invece di conoscere l'hook corretto, la firma della syscall o della funzione kernel, gli indici degli argomenti e i tipi supportati dai programmi eBPF. Il caso della shell è il più evidente: Falco verifica direttamente `proc.tty != 0`, mentre la policy Tetragon basata su `sys_execve` conosce il percorso del binario ma non la semantica della sessione. Per ottenere un comportamento equivalente servirebbero correlazioni o ulteriori segnali, con una complessità maggiore. Tetragon offre in cambio filtraggio vicino alla sorgente e la possibilità di associare azioni di enforcement. La definizione deve quindi bilanciare copertura e selettività: condizioni troppo ampie aumentano falsi positivi e volume di telemetria, mentre condizioni troppo restrittive possono creare punti ciechi.

4.4.2 Risultati del confronto operativo

Nelle Tabelle 4.1 e 4.2 vengono riassunte le principali differenze operative tra Falco e Tetragon, considerando anche aspetti non toccati dalle sezioni precedenti, ma emersi durante le prove di installazione, configurazione e tuning.

Nel complesso, Falco presenta una barriera di ingresso più contenuta quando l'obiettivo iniziale è ottenere visibilità e generare alert. La disponibilità di regole e integrazioni già predisposte facilita l'inserimento in pipeline di monitoring e incident response; rimane tuttavia necessario un periodo di tuning per adattare le regole al comportamento legittimo dei workload.

Tabella 4.1: Confronto Falco e Tetragon: installazione e configurazione.

Aspetto	**Falco**	**Tetragon**
Installazione	Generalmente semplice tramite pacchetti, container o chart Helm. In Kubernetes è normalmente distribuito come DaemonSet.	Normalmente distribuito come agente per nodo, spesso tramite chart Helm. L'installazione è semplice in un ambiente compatibile, ma più sensibile alla configurazione del kernel.
Prerequisiti infrastrutturali	Richiede Linux, accesso agli eventi del kernel e privilegi adeguati. La disponibilità di un driver compatibile o della eBPF probe influenza la procedura di setup.	Richiede Linux con funzionalità eBPF compatibili; il supporto BTF e gli hook disponibili sono particolarmente importanti per il tracing e l'enforcement.
Configurazione iniziale	Di complessità contenuta per il monitoraggio di base, ma può richiedere la selezione della sorgente degli eventi, degli output e delle regole da attivare.	Di complessità più elevata poiché oltre all'agente è necessario definire le *TracingPolicy* che si vuole avere.
Modalità di configurazione	File YAML per le regole e la configurazione dell'agente; supporta output e integrazioni verso numerosi sistemi esterni.	Risorse Kubernetes YAML, come *TracingPolicy* e *ClusterTracingPolicy*, oltre ai parametri dell'agente e del deployment.
Curva di apprendimento delle policy	Sintassi dichiarativa orientata alla condizione dell'evento e al contesto; nel complesso è di facile apprendimento.	Richiede maggiore familiarità con eventi di processo e syscall, hook eBPF e contesto Kubernetes.

Tabella 4.2: Confronto Falco e Tetragon: gestione e comportamento operativo.

Aspetto	**Falco**	**Tetragon**
Tuning e falsi positivi	Il tuning delle regole è spesso necessario per ridurre il rumore; le eccezioni possono essere gestite nella logica delle regole.	Il filtraggio vicino alla sorgente può ridurre la telemetria non necessaria, ma policy troppo ampie o azioni di blocco non verificate possono produrre regressioni operative.
Integrazione	Ampia interoperabilità con syslog, webhook, file, SIEM e pipeline di risposta.	Integrazione naturale con Kubernetes e l'ecosistema Cilium; l'output può essere raccolto e inoltrato attraverso componenti esterni.
Aggiornamento e gestione	[Falco]	[Tetragon]
Enforcement	Orientato alla detection, alerting e risposta successiva all'evento; non permette enforcement diretto.	Progettato per supportare enforcement a bassa latenza, con possibilità di terminare processi o bloccare azioni secondo la policy.

Tetragon richiede invece una preparazione più attenta dell'ambiente, soprattutto quando si utilizzano policy contestualizzate o azioni di enforcement. Questa maggiore complessità iniziale è compensata dalla possibilità di eseguire filtraggio e controllo più vicini all'evento, con una riduzione della finestra di rischio e una granularità elevata sulle operazioni di processo e syscall.

La scelta non è quindi riconducibile a una semplice contrapposizione tra uno strumento "facile" e uno "difficile". Falco è generalmente più adatto a un'adozione progressiva centrata su detection e interoperabilità, mentre Tetragon è più indicato quando il team dispone di competenze Linux di basso livello e il requisito di enforcement giustifica un investimento maggiore nella configurazione e nella validazione delle policy. È perciò da considerare come valida la possibilità di combinare i due strumenti, sfruttando i pro e i contro di ciascuno per ottenere un sistema di runtime security più completo.

Capitolo 5

Testing automatizzato

In questo capitolo vengono descritte le metodologie e gli strumenti utilizzati per la validazione sperimentale di Falco e Tetragon, con l'obiettivo di misurare dati quantitativi e confrontabili.

5.1 Motivazione e Obiettivi

Il confronto tra Falco e Tetragon non può limitarsi a una descrizione architetturale o a un'analisi qualitativa delle funzionalità dichiarate. Entrambi gli strumenti operano su flussi di eventi runtime ad alta frequenza, in cui il comportamento effettivo dipende dal carico, dal tipo di workload, dalla configurazione delle policy e dalle caratteristiche del kernel.

La motivazione sperimentale nasce da tre esigenze. Primo, verificare la capacita reale di individuare comportamenti malevoli, anche in presenza di rumore operativo (processi legittimi, burst di syscall, I/O intenso). Secondo, misurare il costo operativo dell'osservabilita kernel-level, valutandone l'effetto sulle risorse del sistema. Terzo, analizzare la differenza tra un modello prevalentemente reattivo (Falco) e un modello kernel-first con possibilita di blocco (Tetragon), soprattutto in termini di latenza e finestra di rischio.

Gli scenari di test mirano a replicare situazioni reali di minacce e attacchi, considerando classi di vulnerabilita e tecniche offensive realistiche per ambienti cloud-native, tra cui:

- **Accesso non autorizzato a file sensibili:** lettura o modifica di file di configurazione, credenziali o chiavi private.
- **Esecuzione di comandi e *living-off-the-land*:** avvio di shell inattese, catene di comandi sospetti, utilizzo di binari legittimi per finalità malevole.
- **Comportamenti di evasione e overload:** burst di eventi, flood di syscall e pattern rumorosi volti a ridurre la qualita del segnale o a causare perdita di eventi.
- **Ricerca di informazioni:** tentativi di enumerazione del sistema e dei container alla ricerca di possibili vettori di attacco.

L'obiettivo non è dimostrare che uno strumento sia "migliore" in assoluto, ma misurare in quali condizioni ciascun approccio mantiene efficacia, affidabilità e sostenibilità operativa.

5.2 Metriche di valutazione

Dal punto di vista metodologico, uno degli obiettivi centrali di questo progetto è la costruzione di una matrice di confusione per la componente di rilevamento, con classificazione degli eventi in *True Positive* (TP), *False Positive* (FP), *False Negative* (FN) e *True Negative* (TN). A partire da questa base vengono calcolate le seguenti metriche principali:

$$\text{Detection Recall (TPR)} = \frac{TP}{TP + FN}$$

per misurare la percentuale di attacchi o comportamenti malevoli correttamente identificati.

$$\text{Precision} = \frac{TP}{TP + FP}$$

per misurare l'affidabilità degli avvisi generati e perciò il loro impatto sul carico di notifica.

Accanto alle metriche derivate dalla matrice di confusione, il framework include metriche di reattività e di costo infrastrutturale:

$$\text{Detection Latency } (ms)$$

definita come intervallo temporale tra esecuzione del payload e scrittura del log/evento.

$$\text{Resource Overhead} = \{\text{CPU } (\%),\ \text{RAM } (MB)\}$$

per quantificare l'impatto sull'host degli agenti durante scenari di stress.

I risultati ottenuti costituiranno la base del confronto nel Capitolo 6, con l'obiettivo di fornire indicazioni pratiche per la scelta dello strumento più adatto a un determinato contesto operativo.

5.3 Configurazione dell'ambiente di test

L'ambiente sperimentale è stato predisposto in una macchina virtuale dedicata con le seguenti caratteristiche (riportate a titolo di documentazione e riproducibilita):

- Hypervisor: VirtualBox 7.2.2;
- Sistema operativo: Ubuntu 26.04.1 LTS;
- Kernel: Linux 7.0.0-29-generic;
- CPU: 4 vCPU 3.0 GHz;
- RAM: 16 GB DDR4;

La scelta di eseguire i test in una VM consente di isolare il piano sperimentale dal sistema host, ridurre variabili esterne e ripetere in modo controllato le stesse condizioni di carico e configurazione.

Per il deployment è stata adottata una configurazione Docker Compose con due servizi separati, uno per Falco e uno per Tetragon. Questo approccio permette di mantenere parità di contesto di esecuzione (stesso runtime), versionare in modo esplicito immagini e parametri di avvio e riavviare rapidamente l'intero stack sperimentale tra una campagna di test e la successiva.

Una versione ridotta della configurazione utilizzata è riportata nel Listato 5.1.

Listing 5.1: Docker Compose per la creazione dei servizi Falco e Tetragon.

```
services:
  falco:
    image: falcosecurity/falco:0.44.1
    container_name: falco_monitor
    privileged: true
    pid: "host"
    cgroup: "host"
    network_mode: "host"
    volumes:
      - /var/run/docker.sock:/host/var/run/docker.sock
      - /proc:/host/proc:ro
      - /etc:/host/etc:ro
      - /sys/kernel/tracing:/sys/kernel/tracing:ro
      - ./logs:/var/log/falco
    command: >
      /usr/bin/falco
      -o json_output=true
      -o file_output.enabled=true
      -o file_output.keep_alive=true
      -o file_output.filename=/var/log/falco/falco_events.json

  tetragon:
    image: quay.io/cilium/tetragon:v1.7.0
    container_name: tetragon_monitor
    privileged: true
    pid: "host"
    cgroup: "host"
    network_mode: "host"
    volumes:
      - /sys/kernel/btf/vmlinux:/var/lib/tetragon/btf
      - ./logs:/var/log/tetragon
      - ./tetragon-policies:/etc/tetragon/tetragon.tp.d
    command: >
      /usr/bin/tetragon
      --bpf-lib /var/lib/tetragon/
      --export-filename /var/log/tetragon/tetragon_events.json
      --tracing-policy-dir /etc/tetragon/tetragon.tp.d
      --export-allowlist '{"event_set":["PROCESS_KPROBE", "PROCESS_TRACEPOINT", "
          PROCESS_LSM"]}'
```

Da questa configurazione si può notare come nel comando di avvio di Tetragon è presente l'opzione `--export-allowlist`; essa serve a limitare gli eventi scritti nei log ai soli tipi indicati. La sua funzione, ai fini del nostro programma, è quella di rendere il confronto più omogeneo con Falco, che produce un alert quando una regola viene soddisfatta, invece di registrare indiscriminatamente ogni syscall osservata, come invece succede con Tetragon. Limitare l'export agli eventi pertinenti

alle TracingPolicy riduce quindi il rumore del log, il costo di post-processing e il rischio di interpretare come detection eventi generici non associati a una policy attivata.

Si precisa, inoltre, che con questa configurazione entrambi i servizi sono attivi contemporaneamente durante ogni campagna. L'obiettivo è confrontare Falco e Tetragon nello stesso intervallo temporale e sotto lo stesso carico, evitando che differenze tra esecuzioni separate vengano attribuite erroneamente allo strumento.

Dal punto di vista metodologico, questa configurazione privilegia la riproducibilita rispetto alla generalizzazione assoluta: i risultati dipendono dalla versione del kernel, dalle immagini usate e dalle policy attive. Tuttavia, proprio la tracciabilita delle dipendenze (Compose, versioni, mount, flag di export) rende il setup adatto a un confronto robusto e riproducibile nelle medesime condizioni operative.

5.3.1 Selezione dei test

Gli scenari di test realizzati si dividono in tre gruppi: scenari generici, costruiti per verificare le policy definite nell'ambiente, test Atomic Red Team[1] selezionati per la loro associazione diretta a tecniche MITRE ATT&CK[2] e scenari di stress, destinati a valutare il comportamento della telemetria in condizioni di carico elevato.

Gli scenari generici sono stati scelti per coprire operazioni direttamente osservabili dalle policy di Falco e Tetragon. L'obiettivo, tuttavia, non è quello di verificare il rilevamento solo di semplici comandi, ma di vedere come gli strumenti reagiscono a tecnice di evasione o altri comportamenti atti a confondere la detection. Il gruppo comprende l'avvio di una shell interattiva, l'esecuzione di un binario non riconosciuto, la modifica di `/etc/shadow`, l'accesso a file sensibili e una connessione verso un indirizzo esterno. Sono inoltre presenti test per verificare l'appartenenza ad un container, insieme a casi benigni che producono attività realistiche, quali il riavvio di un servizio web, una richiesta HTTPS e il backup di `/etc/`. I casi benigni costituiscono il riferimento per la stima dei falsi positivi e impediscono di valutare la detection esclusivamente su attività malevole.

[1] Atomic Red Team è una libreria di test open source utile per valutare la sicurezza di un sistema. [9]

[2] MITRE ATT&CK è una base di conoscenza di tecniche di attacco. [22]

Come già preannunciato, per aumentare la riproducibilità e la confrontabilità degli scenari offensivi una parte della suite utilizza Atomic Red Team (ART). A tale scopo è stata realizzata un'immagine Docker apposita (Listato 5.2) contenente il binario `goart` del progetto *go-atomicredteam* [1], che permette di eseguire i test di ART in maniera standardizzata. In questo modo l'immagine usata durante la campagna contiene già il programma pronto all'esecuzione, senza richiedere una compilazione all'avvio di ogni test.

Listing 5.2: Costruzione dell'immagine Docker contenente il binario `goart`.

```
FROM alpine:latest AS builder

RUN apk add --no-cache \
    bash \
    curl \
    git \
    go \
    unzip \
    make

RUN git clone https://github.com/activeshadow/go-atomicredteam.git /goart && \
    cd /goart && \
    make bin/goart

FROM alpine:latest

RUN apk add --no-cache \
    bash \
    curl \
    sudo \
    wget \
    python3

COPY --from=builder /goart/bin/goart /usr/local/bin/goart

CMD ["goart"]
```

I test utilizzati nella suite sono perciò direttamente riferiti a tecniche MITRE ATT&CK. L'accesso a `/etc/passwd` e `/etc/shadow` è riferito a `T1003.008`, il masquerading del processo `crond` a `T1036.003`, la raccolta delle informazioni sul sistema a `T1082` e la ricerca della posizione tramite rete a `T1614`.

Infine, i test di stress, basati su `stress-ng` [3], non rappresentano una tecnica avversaria, ma servono a produrre un volume elevato di exec, open, I/O e

allocazioni di memoria, così da valutare la resilienza e le performance dei due strumenti.

5.3.2 Formato della suite di test

Ogni elemento della suite è descritto da un record JSON contenente parametri come: identificativo, nome, categoria, tipo (maligno o benigno), comando da eseguire e immagine del container. È inoltre possibile indicare se il comando deve essere eseguito in una shell, nel caso in cui sia necessario. L'immagine del container è parte del test perché determina dipendenze e contesto di esecuzione; per i test Atomic Red Team, ad esempio, viene usata l'immagine del Listato 5.2.

Il Listato 5.3 riporta alcuni esempi rappresentativi. Per visionare l'intera suite, si rimanda al repository del progetto [6].

Listing 5.3: Esempi di test utilizzati nella campagna sperimentale.

```
[
  {
    "id": "EVASION-SYMLINK",
    "name": "Path Obfuscation via Symlink",
    "category": "Evasion",
    "test_type": "malicious",
    "command": "ln -s /bin/cat /tmp/update && /tmp/update /etc/passwd",
    "shell_wrap": true,
    "container_image": "alpine:latest"
  },
  {
    "id": "NET-EXT-CONN",
    "name": "Suspicious Network Connection to External IP",
    "category": "Network",
    "test_type": "malicious",
    "command": "nc -z 8.8.8.8 53",
    "shell_wrap": false,
    "container_image": "custom/netcat:alpine"
  },
  {
    "id": "BENIGN-NET",
    "name": "Node.js External API HTTPS Request",
    "category": "False_Positive_Testing",
    "test_type": "benign",
    "command": "node -e 'const https = require(\"https\"); https.get(\"https://api
        .github.com\", (res) => { process.exit(0); });'",
    "shell_wrap": true,
    "container_image": "node:20-alpine"
  },
```

```
  {
    "id": "T1614-ART#2",
    "name": "Get geolocation info through IP-Lookup services",
    "category": "Discovery",
    "test_type": "malicious",
    "command": "goart -t T1614 -i 1",
    "shell_wrap": false,
    "container_image": "custom/art:alpine"
  },
  {
    "id": "STRESS-EXEC",
    "name": "Syscall Flood (Exec & Open)",
    "category": "Stress",
    "test_type": "malicious",
    "command": "stress-ng --exec 8 --open 8 --timeout 30s",
    "shell_wrap": false,
    "container_image": "custom/stress-ng:alpine"
  }
]
```

5.3.3 Definizione delle TracingPolicy

Tetragon, a differenza di Falco, non fornisce un insieme predefinito di regole o policy. La definizione delle TracingPolicy è responsabilità dell'utente ed è stato perciò necessario progettare un insieme di policy mirate agli scenari di test scelti. Il criterio di progettazione è stato mantenere un numero contenuto di hook e condizioni specifiche, così da limitare i falsi positivi e il volume di telemetria, evitando allo stasso tempo di creare punti ciechi. Le policy realizzate sono quindi mirate a operazioni osservabili e interpretabili, come l'esecuzione di shell, la scrittura in directory sensibili, la modifica delle capability, le connessioni TCP e l'esecuzione da percorsi scrivibili.

Nel perimetro di questa valutazione, Tetragon è stato utilizzato in modalità osservativa: non sono state configurate azioni di enforcement che terminassero i processi o modificassero il comportamento dei test. In questo modo entrambi gli strumenti osservano gli stessi workload senza che Tetragon alteri il campione sperimentale o impedisca a Falco di ricevere gli eventi successivi.

Un esempio di policy per la rilevazione di connessioni TCP è riportato nel Listato 5.4.

Listing 5.4: TracingPolicy per la rilevazione della connessione TCP.

```yaml
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "detect-tcp-connection"
spec:
  kprobes:
  - call: "tcp_connect"
    syscall: false
    args:
    - index: 0
      type: "sock"
    selectors:
      - matchPIDs:
          - isNamespacePID: true
            operator: "NotIn"
            values: [0]
```

È importante sottolineare che, per lo scopo di questa tesi, le TracingPolicy sono state progettate per essere il più specifiche possibili; tuttavia, nei casi in cui la selettività potesse comportare perdite di copertura, si è deciso di adottare un approccio più ampio, accettando un aumento del rumore. In un contesto reale sarebbe opportuno effettuare audit e tuning delle policy per adattarle alle esigenze specifiche.

5.4 Progettazione del programma di testing

In questa sezione vengono descritte le scelte metodologiche e le motivazioni alla base della progettazione del programma di test, ponendo anche l'accento sulle considerazioni relative all'implementazione e all'uso degli strumenti di telemetria.

5.4.1 Architettura

Il programma di testing è stato progettato come una pipeline automatizzata per l'esecuzione degli scenari di test, la raccolta della telemetria prodotta dagli strumenti di sicurezza e il calcolo delle metriche di valutazione. L'architettura è organizzata secondo una separazione delle responsabilità: il modulo principale coordina il flusso di esecuzione, il `Loader` interpreta la suite di test, il `Runner` gestisce il ciclo di vita dei container e i moduli `TelemetryCollector` e `PerformanceMonitor`

acquisiscono rispettivamente gli eventi di sicurezza e le statistiche di utilizzo delle risorse.

La sequenza complessiva delle interazioni tra i componenti è riportata nella Figura 5.1, mentre il diagramma delle classi nella Figura 5.2 mostra l'organizzazione delle classi del programma. Il diagramma di sequenza, in particolare, evidenzia il ruolo di `main` come coordinatore; infatti, è sua responsabilità avviare i moduli di telemetria, ciclare attraverso la suite di test, aggregare i risultati e infine inviarli al modulo `Evaluator` per il calcolo delle metriche.

Il `TelemetryCollector` viene avviato come attività asincrona prima dell'esecuzione dei test: ciascuno dei suoi task osserva il rispettivo file di log e rimane in attesa di nuove righe senza caricare nuovamente il contenuto già presente. Questa scelta consente di distinguere gli eventi generati durante la campagna corrente da quelli eventualmente prodotti in precedenza. Per ogni riga acquisita, il collector deserializza l'evento JSON, estrae il contesto del container e costruisce un oggetto `SecurityEvent` con sorgente, timestamp, identificativo del container, nome della policy che ha generato l'evento e contenuto originale dell'evento.

L'esecuzione di un singolo test inizia con la lettura dei contatori degli eventi persi esposti da Prometheus[3]. Il modulo `PerformanceMonitor` interroga l'endpoint HTTP dell'istanza locale, passando una query PromQL e interpretando il valore numerico restituito nella risposta JSON. Per Tetragon viene interrogata la serie `tetragon_observer_ringbuf_events_lost_total`, mentre per Falco viene usata `falcosecurity_scap_n_drops_total`. Si tratta di contatori cumulativi: il programma legge il valore iniziale prima dell'esecuzione e quello finale dopo la finestra di raccolta, quindi calcola il delta. Il delta, perciò, rappresenta gli eventi persi durante il singolo test e non il totale storico dall'avvio dei servizi. [8]

L'implementazione in Python di questo meccanismo è illustrato nel Listato 5.5.

[3]Prometheus è un sistema di monitoraggio e alerting open-source. [7]

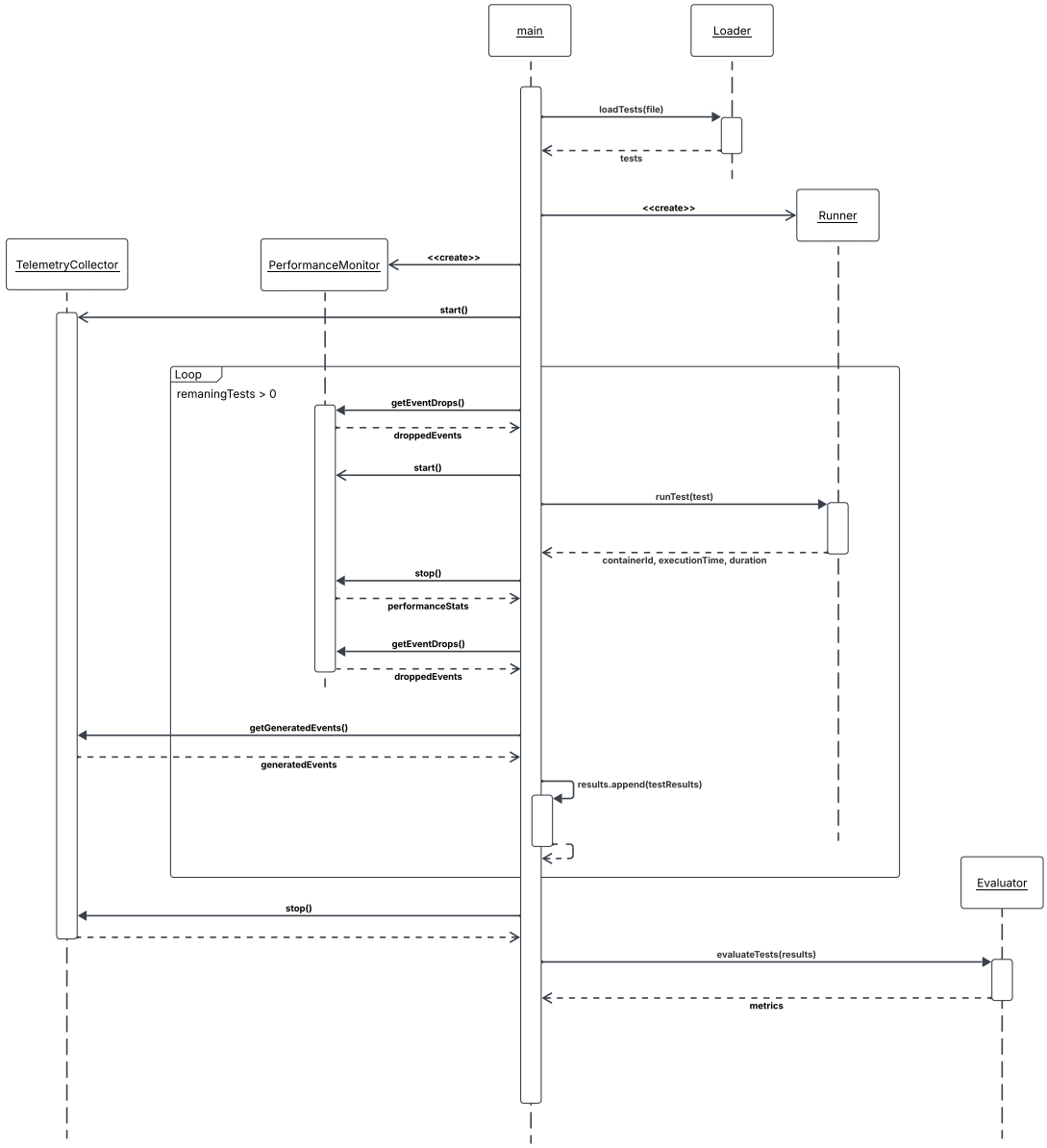


Figura 5.1: Sequenza di esecuzione del programma di testing automatizzato.

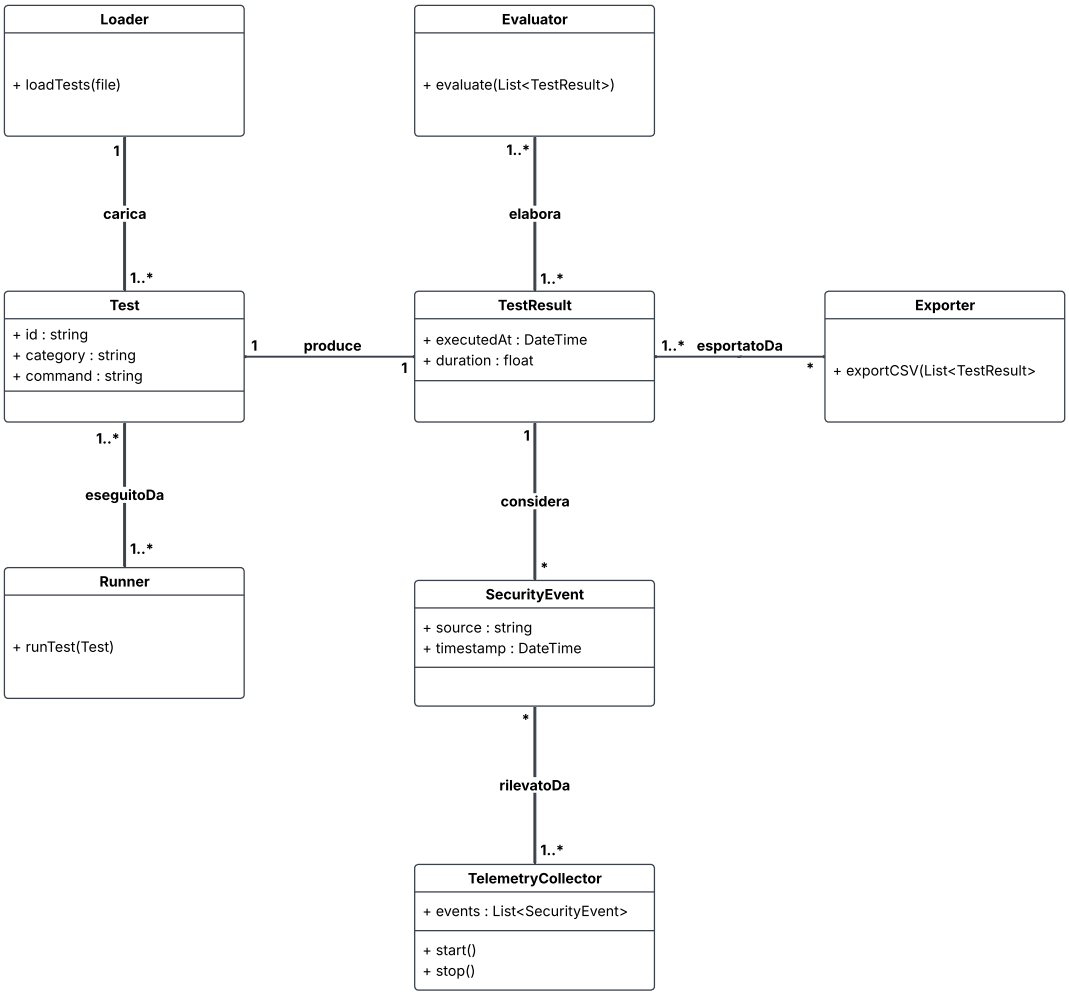


Figura 5.2: Diagramma delle classi del programma di testing automatizzato.

Listing 5.5: Interrogazione di Prometheus per i contatori degli eventi persi.

```python
# Other imports...
import requests

PROMETHEUS_URL = "http://localhost:9090/api/v1/query"

class PerformanceMonitor:
    # Other methods...

    @staticmethod
    def get_drop_metrics() -> dict:
        def query(q: str) -> int:
            try:
                res = requests.get(PROMETHEUS_URL, params={'query': q}).json()
                data = res.get('data', {}).get('result', [])
                return int(data[0]['value'][1])
            except Exception as e:
                logger.error(f"Failed to query Prometheus for '{q}': {e}")
                return 0

        return {
            "tetragon": query('tetragon_observer_ringbuf_events_lost_total'),
            "falco": query('falcosecurity_scap_n_drops_total')
        }
```

Successivamente il monitor delle prestazioni avvia un thread che, a intervalli regolari, interroga Docker per ottenere le statistiche dei container in cui sono in esecuzione Falco e Tetragon. In particolare, vengono raccolti l'utilizzo della CPU e la memoria allocata; al termine del test tali campioni sono aggregati calcolando la CPU media e il valore massimo della memoria osservata.

Il `Runner` crea quindi un container a partire dall'immagine indicata nella definizione del test. Se richiesto dal campo `shell_wrap`, il comando viene eseguito attraverso `/bin/sh -c`; in caso contrario viene passato direttamente al container. L'istante di avvio viene ricavato dagli attributi Docker, dopodiché il runner attende la terminazione del container, ne misura la durata complessiva e lo rimuove.

Una volta terminata l'esecuzione, il programma lascia trascorrere una breve finestra temporale per consentire ai `TelemetryCollector` di acquisire gli eventi generati per poi interrompere il `PerformanceMonitor`. Gli eventi raccolti vengono filtrati confrontando l'identificativo del container[4] che li ha generati e aggregati

[4]Per capire meglio il perché di questa scelta, si rimanda alla Sezione 5.4.2

dentro a un oggetto `TestResult`, assieme alle statistiche di prestazione. Il ciclo prosegue fino all'esaurimento della suite o all'interruzione manuale.

Al termine della campagna, l'insieme dei risultati viene passato al modulo `Evaluator`. Per ciascuno strumento, l'evaluator considera rilevato un test quando è presente almeno un `SecurityEvent` corrispondente. Il confronto con il tipo dichiarato nel test consente di aggiornare la matrice di confusione e di calcolare Recall, Precision e latenza media di rilevazione[5]. La latenza è ottenuta come differenza tra il timestamp del primo evento associato e l'istante di avvio del container.

Infine, il modulo `Exporter` serializza in formato CSV sia i dati relativi ai singoli test sia le metriche aggregate. Il primo blocco del file contiene durata, rilevamento da parte di ciascuno strumento, consumo di risorse e perdita di eventi; il secondo riporta i valori TP, FP, TN, FN, Recall, Precision e latenza media. La separazione tra dati grezzi e metriche aggregate conserva la possibilità di effettuare analisi successive senza dover ripetere la campagna sperimentale.

5.4.2 Isolamento dei test e correlazione degli eventi

Ogni scenario viene eseguito all'interno di un container usa-e-getta, creato prima dell'avvio e rimosso al termine. Questa scelta impedisce che file creati, processi residui, modifiche al filesystem o altri effetti collaterali di un test influenzino quello successivo. L'isolamento è particolarmente importante quando una campagna alterna test malevoli, casi benigni e workload di stress: senza una nuova istanza, un processo lasciato attivo o un artefatto temporaneo potrebbe produrre eventi attribuiti al test sbagliato.

Il container usa-e-getta semplifica inoltre la correlazione tra test eseguito ed eventi osservati. Al momento della creazione, Docker assegna al container un identificativo univoco; il `Runner` restituisce tale ID per essere integrato nel `TestResult`. Il `TelemetryCollector`, invece, preleva e normalizza l'ID del container dagli eventi generati da Falco e Tetragon. In questo modo ogni evento può essere associato al test corretto, anche se i due strumenti producono log in formati diversi e con campi differenti.

[5]Le metriche considerate e la loro definizione si possono trovare nella Sezione 5.2

Sono state considerate anche strategie alternative, come la generazione di uno UUID che sarebbe stato integrato all'interno del comando stesso in forma di variabile d'ambiente. Tale soluzione è stata tuttavia abbandonata poiché richiedeva una maggiore complessità nel recuperare l'UUID dai log e non garantiva la stessa robustezza. Il container ID offre invece una chiave generata dal runtime, stabile per l'intera esecuzione e condivisa dai due flussi di telemetria.

Capitolo 6

Risultati e Discussione

Questo capitolo presenta i risultati ottenuti dall'esecuzione di due campagne di test. La prima comprende gli scenari generici, i casi benigni e i test Atomic Red Team, mentre la seconda è composta da test mirati a valutare la performance di Falco e Tetragon in situazioni di stress. Il confronto considera quattro dimensioni: efficacia della detection, falsi positivi, latenza e costo computazionale. I risultati sono interpretati tenendo conto del perimetro sperimentale: Tetragon è stato utilizzato in modalità osservativa, senza policy di enforcement, e i due strumenti erano attivi contemporaneamente.

6.1 Risultati della suite senza stress

La suite utilizzata contiene 22 test, di cui 18 classificati come malevoli e 4 come benigni. Inoltre, 12 degli scenari sono stati realizzati ad hoc, mentre 10 sono stati selezionati da Atomic Red Team. Per maggiori informazioni sulla composizione della suite, si rimanda alla Sezione 5.3.1. Per la lista completa dei test, si rimanda al repository del progetto [6].

6.1.1 Efficacia della detection

La Tabella 6.1 riassume la matrice di confusione e le metriche derivate. Falco rileva 2 dei 18 test malevoli, mentre Tetragon ne rileva 14. Ne risulta un Recall

rispettivamente pari a 0,1111 e 0,7778. In termini relativi, Tetragon riconosce circa il 77,8% degli scenari malevoli considerati, contro l'11,1% di Falco.

Tabella 6.1: Risultati di detection sulla suite `all-no-stress`.

Strumento	**TP**	**FP**	**TN**	**FN**	**Recall**	**Precisione**
Falco	2	1	3	16	0,1111	0,6667
Tetragon	14	4	0	4	0,7778	0,7778

Il recall più elevato di Tetragon indica una copertura maggiore sugli scenari selezionati, ma non implica che il risultato sia generalizzabile a ogni tecnica o configurazione. Infatti, le policy attive determinano quali eventi vengono rilevati e associati a un test. La precisione è pari a 0,7778 per Tetragon e 0,6667 per Falco: entrambi producono falsi positivi, ma il numero assoluto è diverso. In particolare, Tetragon genera quattro rilevazioni su casi benigni e nessun vero negativo, mentre Falco genera una rilevazione erronea, lasciando tre casi benigni senza avvisi. Il risultato evidenzia quindi un trade-off tra copertura e rumore: Tetragon intercetta più scenari malevoli, ma nel setup considerato segnala anche tutte le attività benigne della suite.

6.1.2 Latenza di rilevazione

La latenza è calcolata come intervallo tra l'avvio del container (e quindi del comando da eseguire) e il timestamp riportato dal primo evento generato dal test. Considerando esclusivamente i test rilevati, la latenza media è di 268,844 ms per Falco e 254,704 ms per Tetragon. Tetragon presenta quindi un vantaggio medio di circa 14,140 ms, pari a circa il 5,3%. Nonostante la differenza sia contenuta, il risultato è coerente con l'architettura di Tetragon, che esegue il filtraggio e la valutazione delle policy più vicino al kernel, riducendo la finestra di latenza tra evento e rilevazione.

6.1.3 Costo computazionale

Per la CPU viene riportata la media dei campioni raccolti, mentre per la memoria viene riportato il valore massimo osservato. Le due metriche descrivono aspetti

differenti: la prima evidenzia il lavoro di processamento sostenuto dal monitor, mentre la seconda rappresenta il picco di memoria occupata.

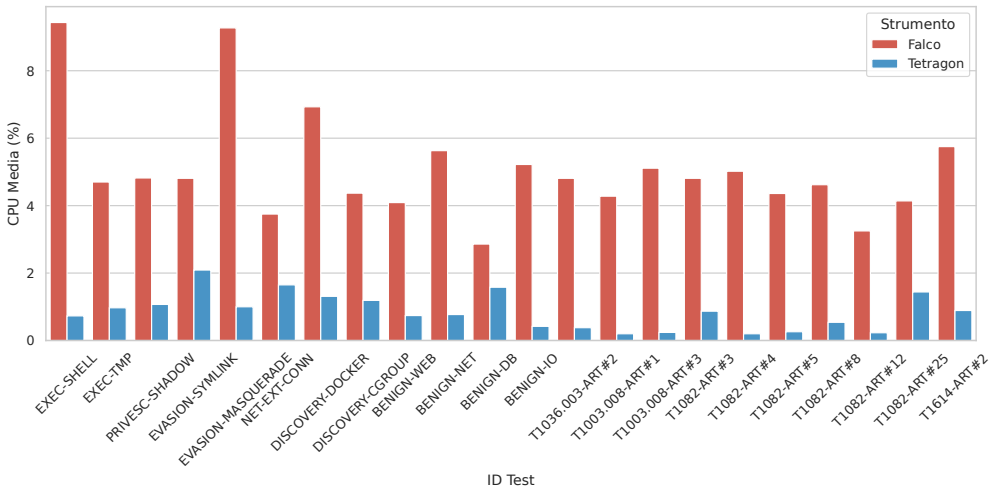


Figura 6.1: Utilizzo medio della CPU nella suite `all-no-stress`.

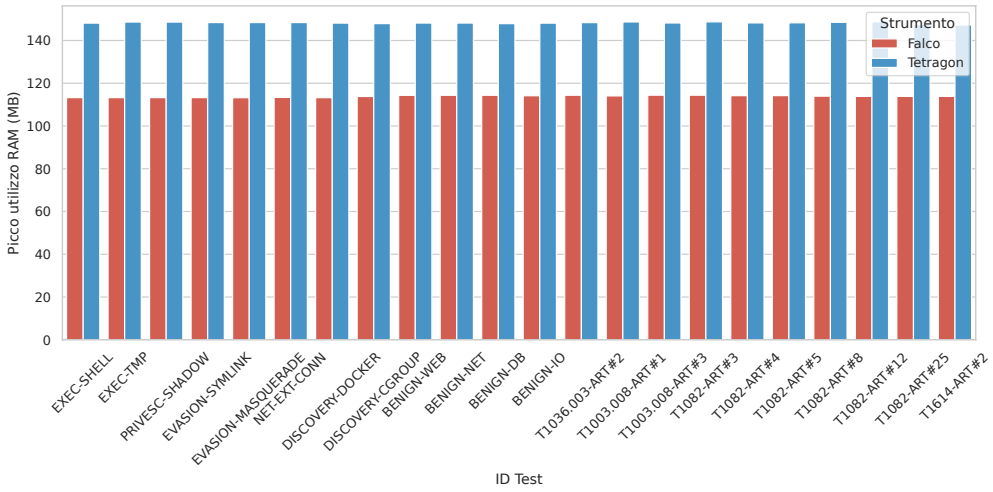


Figura 6.2: Utilizzo massimo della memoria nella suite `all-no-stress`.

Nella suite senza stress, Falco utilizza mediamente il 5,62% della CPU, contro lo 0,92% di Tetragon. Il valore di Falco è quindi circa sei volte maggiore. Il comportamento si inverte per la memoria: Falco raggiunge 113,283 MB, mentre Tetragon raggiunge 147,768 MB. Tetragon utilizza quindi circa 34,5 MB in più,

pari a un incremento di circa il 30,5% rispetto al valore osservato per Falco. Le Figure 6.1 e 6.2 rendono visibile questo compromesso: Falco richiede più lavoro di CPU per la pipeline di acquisizione e valutazione, mentre Tetragon mantiene un consumo di CPU inferiore a fronte di una maggiore occupazione di memoria.

Questi valori non devono essere interpretati come una misura assoluta del costo dei due strumenti. Sono riferiti alle versioni, alle policy, al kernel e al carico della macchina virtuale usati nell'esperimento. Sono tuttavia utili per il confronto controllato, poiché entrambi i monitor sono stati eseguiti nello stesso ambiente e nello stesso intervallo dei test.

6.2 Risultati della suite di stress

La campagna di stress comprende due scenari: `STRESS-EXEC`, basato su un'elevata frequenza di operazioni di esecuzione e apertura di file, e `STRESS-IO-RAM`, orientato a I/O e allocazioni di memoria. Entrambi i test durano alcune decine di secondi e non sono stati progettati per verificare Recall e Precision degli strumenti, ma bensì per sottoporre le loro pipeline di telemetria a condizioni di carico.

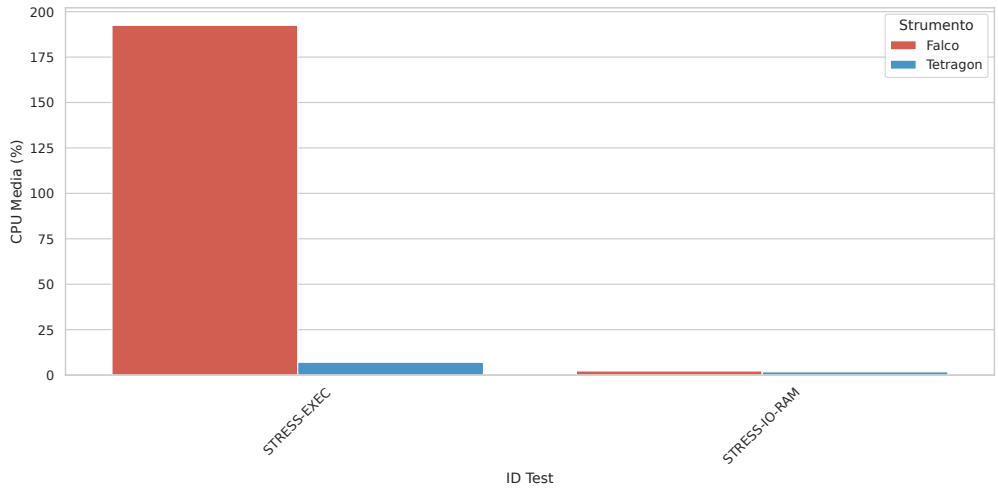


Figura 6.3: Utilizzo medio della CPU durante i test di stress.

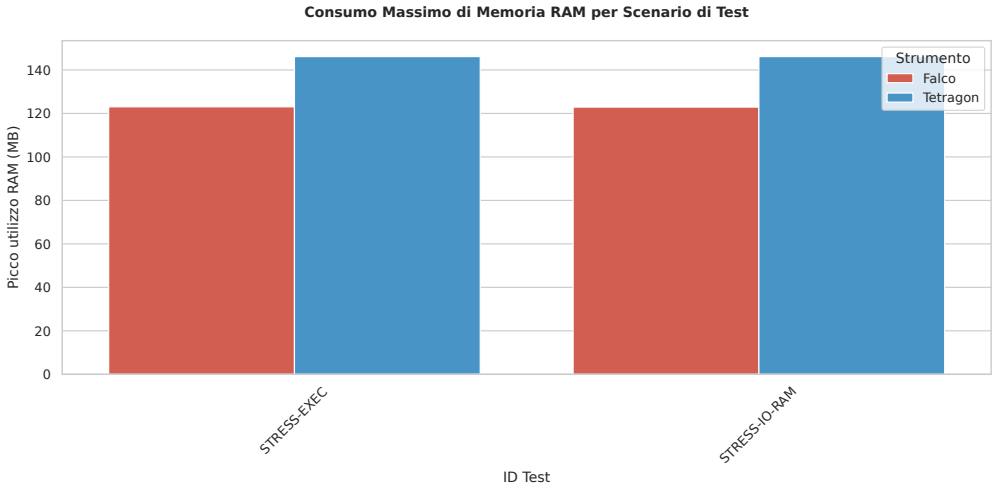


Figura 6.4: Utilizzo massimo della memoria durante i test di stress.

Nel test `STRESS-EXEC`, Falco raggiunge un utilizzo medio della CPU pari al 192,49%[1], mentre Tetragon si mantiene al 7,06%. Nel test `STRESS-IO-RAM`, invece, l'utilizzo medio risulta analogo agli scenari precedenti. Il comportamento di Falco dipende quindi fortemente dalla natura del carico: la pressione su exec e open produce un costo di processamento molto più alto rispetto al carico combinato di I/O e RAM.

Per quanto riguarda il test sulla memoria, il comportamento è più stabile. Falco raggiunge 123,03 MB in `STRESS-EXEC` e 122,91 MB in `STRESS-IO-RAM`; Tetragon raggiunge rispettivamente 146,15 MB e 146,16 MB. Il risultato conferma il compromesso già osservato nella suite senza stress: Tetragon utilizza più memoria, mentre Falco è più sensibile in termini di CPU quando il numero di eventi aumenta rapidamente. Le Figure 6.3 e 6.4 mostrano questo comportamento nei due scenari.

6.2.1 Perdita di eventi

Il risultato più significativo riguarda il contatore degli eventi persi. Infatti, dopo diverse iterazione del test `STRESS-EXEC`, Falco registra in media 1.980.597 eventi

[1]Il valore superiore al 100% è compatibile con una misura aggregata su più CPU della macchina virtuale e indica un carico molto elevato per il monitor.

persi, mentre Tetragon ne registra zero. Durante `STRESS-IO-RAM`, invece, entrambi i contatori rimangono a zero, rafforzando l'ipotesi che gli strumenti risentano meno della carenza di memoria. Il confronto è riportato nella Figura 6.5.

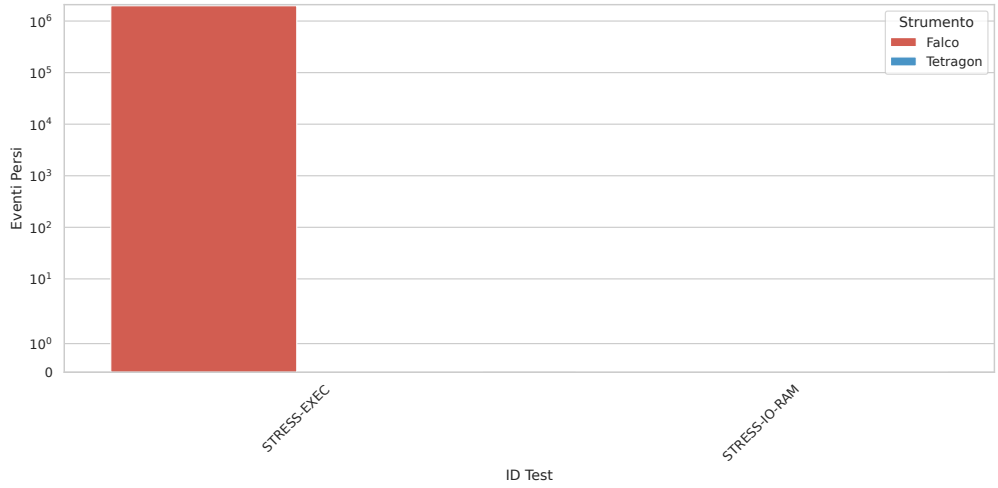


Figura 6.5: Eventi persi rilevati durante i test di stress.

Il completo sovraccarico di Falco in `STRESS-EXEC` non è inatteso alla luce delle differenze architetturali tra i due strumenti. Falco deve acquisire un flusso molto intenso di eventi, trasferirlo verso il componente user-space, applicare la logica di valutazione delle regole e serializzare gli alert. Quando la velocità di produzione supera la capacità di consumo della pipeline, il buffer di acquisizione può saturarsi e gli eventi successivi venire persi. Il dato osservato è coerente con il picco di CPU misurato nello stesso test.

Tetragon, invece, filtra gli eventi vicino alla sorgente mediante programmi eBPF e produce l'output in funzione delle policy attive. Nel setup adottato, l'allowlist limita inoltre gli eventi esportati ai tipi rilevanti per il confronto. Il minor volume trasferito a user-space riduce la pressione sulla fase di serializzazione e rende plausibile l'assenza di drop osservati. Questo risultato non dimostra che Tetragon sia immune alla perdita di eventi in ogni condizione: indica che, nel carico e nella configurazione sperimentati, la sua pipeline ha mantenuto la capacità di assorbire gli eventi prodotti.

Per mitigare il problema, Falco può essere configurato per generare un avviso quando il meccanismo di acquisizione segnala una saturazione. Tale avviso è utile

per rendere esplicita la perdita di visibilità e per attivare una risposta operativa, ma non risolve direttamente il problema: informa che gli eventi sono già stati persi e non identifica necessariamente quali azioni abbiano causato la saturazione. L'alert può quindi supportare il monitoraggio della salute del sistema, ma non consente da solo di applicare un intervento mirato sul workload responsabile, né di ricostruire gli eventi mancanti.

6.3 Discussione complessiva

I risultati mostrano due profili operativi distinti. Falco presenta un consumo di memoria inferiore e mantiene una precisione non trascurabile sui casi della suite, ma il Recall è ridotto e il costo CPU cresce drasticamente sotto un flusso intenso di eventi. Tetragon raggiunge una copertura più elevata e una latenza media leggermente inferiore, utilizzando però più memoria e generando più falsi positivi nel campione analizzato. La sua maggiore robustezza nel test `STRESS-EXEC` è il risultato più netto della sperimentazione, ma deve essere letto insieme alla configurazione delle policy e dell'export utilizzata.

Non emerge quindi un vincitore assoluto; Falco rimane adatto a scenari in cui la generazione di alert, l'integrazione con output user-space e il contenimento della memoria sono prioritari, purché siano monitorate la capacità di elaborazione e la possibile perdita di eventi. Tetragon risulta più efficace quando la priorità è filtrare gli eventi vicino al kernel e mantenere la telemetria sotto carico, accettando un maggiore consumo di memoria e la necessità di un tuning più accurato delle policy per ridurre i falsi positivi.

Un ulteriore elemento di cautela riguarda la configurazione delle policy. Falco è stato eseguito con il set di regole predefinito, mentre per Tetragon sono state definite TracingPolicy mirate agli scenari di test, poiché esso non dispone di un insieme equivalente di policy di detection già attivo nel setup utilizzato. Non è stata quindi realizzata una replica completa dell'intero set di regole Falco. Il confronto misura pertanto il comportamento di due configurazioni sperimentali concrete e non l'efficacia massima teorica dei due strumenti.

I valori di Recall, Precision e latenza devono quindi essere considerati indicativi. Un confronto più esteso dovrebbe prevedere il tuning indipendente delle regole

Falco e delle policy Tetragon, la definizione di una copertura comune delle tecniche e scenari ripetuti con diverse soglie di selettività. In particolare, il tuning potrebbe aumentare la copertura di Falco o ridurre i falsi positivi di Tetragon, modificando il bilanciamento osservato nella campagna attuale.

Capitolo 7

Conclusione e sviluppi futuri

Il presente lavoro ha analizzato il ruolo della sicurezza a livello kernel negli ambienti cloud-native, confrontando Falco e Tetragon come strumenti per l'osservazione e la rilevazione di comportamenti sospetti nei container. L'obiettivo principale era comprendere in che modo i due approcci differissero nella capacità di individuare attività anomale e quali compromessi emergessero in termini di precisione, latenza e complessità di configurazione.

L'analisi teorica ha evidenziato una differenza architetturale fondamentale. Falco adotta un modello principalmente orientato alla detection e all'alerting: acquisisce gli eventi, li arricchisce con il contesto disponibile e valuta le regole in user-space. Tetragon, invece, utilizza un approccio kernel-first, basato su programmi eBPF, hook del kernel e policy configurabili, con la possibilità di filtrare gli eventi più vicino alla loro sorgente e di supportare meccanismi di enforcement. Questa distinzione si riflette anche nella configurazione: le regole Falco risultano generalmente più immediate da definire e integrare, mentre le TracingPolicy di Tetragon richiedono una conoscenza più approfondita degli hook, degli argomenti delle syscall e del contesto del kernel.

Il confronto sperimentale ha confermato che non esiste uno strumento migliore in assoluto, ma che la scelta dipende dalle priorità operative. Nella suite senza stress, Tetragon ha rilevato 14 dei 18 scenari malevoli, raggiungendo un Recall pari a 0,7778, mentre Falco ne ha rilevati 2, con un Recall pari a 0,1111. Tetragon ha quindi offerto una copertura maggiore nel perimetro considerato, a fronte di

un numero più elevato di falsi positivi. Falco ha invece mantenuto un consumo massimo di memoria inferiore, mentre Tetragon ha mostrato un utilizzo medio della CPU più contenuto. La latenza media è risultata leggermente più bassa per Tetragon, ma la differenza osservata non costituisce da sola un elemento decisivo.

La differenza più significativa è emersa durante il test di stress basato su un'elevata frequenza di operazioni di esecuzione e apertura di file. In tale scenario Falco ha raggiunto un utilizzo della CPU molto elevato e ha registrato una consistente perdita di eventi, mentre Tetragon non ha registrato eventi persi nella configurazione adottata. Questo risultato è coerente con la diversa organizzazione delle pipeline: Falco deve trasferire gli eventi verso lo user-space e valutarli attraverso il motore di regole, mentre Tetragon può ridurre il volume esportato mediante filtraggio vicino alla sorgente. Il risultato non implica tuttavia l'assenza di limiti per Tetragon, poiché dipende dalle policy, dall'allowlist di esportazione, dal carico e dall'ambiente sperimentale.

Alla domanda di ricerca si può quindi rispondere affermando che Falco e Tetragon realizzano forme diverse di runtime security. Falco è più adatto quando sono prioritari semplicità di adozione, flessibilità delle regole, alerting e integrazione con sistemi esterni. Tetragon risulta preferibile quando sono richieste una maggiore vicinanza al kernel, una copertura più elevata sugli eventi selezionati e una gestione più efficiente dei flussi ad alta frequenza, accettando una maggiore complessità di configurazione e un consumo di memoria superiore. I risultati mostrano dunque un trade-off concreto tra facilità di gestione, copertura, rumore e costo delle risorse, anziché una superiorità generalizzabile di uno dei due strumenti.

La portata dei risultati deve essere interpretata alla luce dei limiti della ricerca. Gli esperimenti sono stati eseguiti in una singola macchina virtuale, con specifiche versioni del kernel, delle immagini e dei componenti utilizzati. Inoltre, Falco è stato valutato con il set di regole predefinito, mentre per Tetragon sono state definite policy mirate agli scenari della suite; non è stata quindi realizzata una copertura perfettamente equivalente. Tetragon è stato utilizzato in modalità osservativa, senza valutare sperimentalmente l'effetto delle azioni di enforcement. Di conseguenza, le metriche di Recall, Precision e latenza descrivono il comportamento delle configurazioni adottate e non rappresentano una misura assoluta delle capacità teoriche dei due progetti.

Gli sviluppi futuri dovrebbero partire da questi limiti, realizzando un tuning indipendente e sistematico delle regole Falco e delle policy Tetragon, definendo una copertura comune delle tecniche e ripetendo gli scenari con diversi livelli di selettività. Sarebbe inoltre utile estendere la valutazione a più ambienti e considerare direttamente le modalità di enforcement di Tetragon, mantenendo invariati i criteri di confronto. In questo modo sarebbe possibile verificare se il vantaggio osservato nella copertura e nella gestione dei carichi elevati rimanga stabile anche al variare del contesto e se il costo aggiuntivo di configurazione produca benefici concreti nei sistemi di produzione.

Appendice A

Dichiarazione sull'uso di strumenti di GenAI

Questa appendice documenta l'impiego di strumenti di intelligenza artificiale generativa durante la redazione della tesi. Gli strumenti sono stati utilizzati come supporto alla scrittura del codice, alla revisione linguistica e alla riorganizzazione di alcuni passaggi testuali. La responsabilità per la selezione delle fonti, la correttezza delle descrizioni tecniche, l'esecuzione degli esperimenti e l'interpretazione dei risultati rimane dell'autore.

A.1 Strumenti utilizzati

Sono stati utilizzati GitHub Copilot e Gemini 3.1 Pro. Il loro impiego ha riguardato principalmente il supporto alla scrittura e alla revisione del codice della suite di testing, il controllo della chiarezza espositiva e l'individuazione di possibili errori linguistici. Le regole eBPF, i comandi di test, le misurazioni e i risultati riportati nella tesi sono stati verificati nell'ambiente sperimentale descritto nel Capitolo 5.

A.2 Esempi di Prompt

Le richieste hanno riguardato la proposta di implementazioni per il programma di testing, la revisione di frammenti LaTeX e Python, il controllo della coerenza

tra descrizione metodologica e risultati e la riformulazione di passaggi tecnici. Le risposte generate sono state usate come supporto, sottoposte a verifica manuale e adattate al contenuto effettivamente prodotto durante il lavoro.

Bibliografia

[1] Active Shadow. go-atomicredteam. https://github.com/activeshadow/go-atomicredteam.

[2] The Cilium Authors. Cilium - cloud native, ebpf-based networking, observability, and security. https://cilium.io/.

[3] Colin Ian King. ColinIanKing/stress-ng — GitHub. https://github.com/ColinIanKing/stress-ng.

[4] eBPF.io authors. What is eBPF? An Introduction and Deep Dive into the eBPF Technology. https://ebpf.io/what-is-ebpf/.

[5] Ivan Velichko. Exploring Tetragon - A Security Observability Tool for Kubernetes, Docker, and Linux. https://labs.iximiuz.com/tutorials/introduction-to-tetragon.

[6] Matteo Grandini. Test Harness Thesis. https://github.com/Jackmo04/Test-Harness-Thesis.

[7] Prometheus Authors. Prometheus. https://prometheus.io/.

[8] Prometheus Authors. Querying Prometheus. https://prometheus.io/docs/prometheus/latest/querying/basics/.

[9] Red Canary. Atomic Red Team. https://github.com/redcanaryco/atomic-red-team.

[10] The Falco Authors. Event Sources. https://falco.org/docs/concepts/event-sources/.

[11] The Falco Authors. Extend Falco outputs with falcosidekick. https://falco.org/blog/extend-falco-outputs-with-falcosidekick/.

[12] The Falco Authors. Falco Rules. https://falco.org/docs/concepts/rules/.

[13] The Falco Authors. Falco Use Cases. https://falco.org/about/use-cases/.

[14] The Falco Authors. Kernel Events — Falco. https://falco.org/docs/concepts/event-sources/kernel/.

[15] The Falco Authors. Setup — Falco. https://falco.org/docs/setup/.

[16] The Falco Authors. Supported Events. https://falco.org/docs/reference/rules/supported-events/.

[17] The Falco Authors. The Falco Project. https://falco.org/docs/.

[18] The Falco Authors. Try Falco with Docker. https://falco.org/docs/setup/container/.

[19] The Falco Authors. What is Falco? https://falco.org/about/.

[20] The kernel development community. BPF Type Format (BTF). https://docs.kernel.org/bpf/btf.html.

[21] The Kubernetes Authors. Minikube Documentation. https://minikube.sigs.k8s.io/docs/.

[22] The MITRE Corporation. MITRE ATT&CK. https://attack.mitre.org/.

[23] The Tetragon Authors. Installation and Configuration — Tetragon. https://tetragon.io/docs/installation/.

[24] The Tetragon Authors. Tetragon Documentation. https://tetragon.io/docs/.

[25] The Tetragon Authors. Tetragon Documentation. https://tetragon.io/docs/concepts/tracing-policy/hooks/.

[26] The Tetragon Authors. Tetragon Installation FAQ. https://tetragon.io/docs/installation/faq/.